

Name: Kashinath Sangram Hale

PRN no.: 202501040055

Division: CS-1 Batch:

C13 Course: EDS

Essential of Data Science Laboratory

Practical 3:

CODETANTRA

Home Learn Anywhere

202501040055@mitaoe.ac.in Support Logout

3.1.1.1. Numpy Array Operations

11:01

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyarr...

Submit

```
1 import numpy as np
2
3 # Read the first line for rows and columns
4 rows, cols = map(int, input().split())
5
6 # Read the elements row by row
7 elements = []
8 # for _ in range(rows):
9 #     elements.extend(list(map(int, input().split())))
10
11 elements = np.array([list(map(int, input().split())) for i in range(rows)])
12
13 # Create the NumPy array and reshape it
14 arr = elements.reshape(rows, cols)
15
16 # Output the required information
17 print(arr)
18 print(arr.ndim)
19 print(arr.shape)
20 print(arr.size)
```

Average time
0.018 s
18.50 ms

Maximum time
0.029 s
29.00 ms

2 out of 2 shown test case(s) passed
10 out of 10 hidden test case(s) passed

Test case 1 29 ms

Debug

Expected output

Actual output

3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8

Terminal Test cases

< Prev

Reset

Submit

Next >

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a · matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

matrixOp...

```
11 # Addition
12 add= matrix_a + matrix_b
13 print("Addition (A + B):")
14 print(add)
15 # Subtractio
16 sub= matrix_a-matrix_b
17 print("Subtraction (A - B):")
18 print(sub)
19
20 # Multiplication (element-wise)
21 mul= matrix_a*matrix_b
22 print("Element-wise Multiplication (A * B):")
23 print(mul)
24 # Matrix multiplication (dot product)
25 mdot= np.dot(matrix_a,matrix_b)
26 print("A dot B:")
27 print(mdot)
```

Average time 0.038 s 38.25 ms Maximum time 0.044 s 44.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 44 ms Debug

Expected output	Actual output
Enter Matrix A: 1 2 3 4 5 6 7 8 9	Enter Matrix A: 1 2 3 4 5 6 7 8 9
Enter Matrix B: 1 1 1	Enter Matrix B: 1 1 1

Terminal Test cases

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 a= np.hstack((arr1,arr2))
12 print("Horizontal Stack:")
13 print(a)
14 # Perform vertical stacking (vstack)
15 b=np.vstack((arr1,arr2))
16 print("Vertical Stack:")
```

Average time 0.030 s 30.50 ms Maximum time 0.036 s 36.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 36 ms Debug

Expected output	Actual output
Enter Array1: 1 2 3 4 5 6 7 8 9	Enter Array1: 1 2 3 4 5 6 7 8 9
Enter Array2: 4 5 6	Enter Array2: 4 5 6

Terminal Test cases

3.2.5. Numpy: Copying and Viewing Arrays

18.02

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: `<original_array>`
View array: `<view_array>`

- After modifying the copy:

Original array after modifying copy: `<original_array>`
Copy array: `<copy_array>`

Sample Test Cases

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
```

Average time

0.010 s

10.25 ms

Maximum time

0.014 s

14.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 14 ms

Debug

Expected output

10 20 30 40 50 60 70 80

Actual output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Terminal

Test cases

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

20.45

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Sample Test Cases

arrayOpe...

```
9 broadcast_value = int(input("Value to add: "))
10 # 1. Searching: Find indices
11 indices = np.where(array1 == search_value)[0]
12 print(indices)
13
14 # 2. Counting occurrences
15 count = np.count_nonzero(array1 == count_value)
16 print(count)
17
18 # 3. Broadcasting addition
19 broadcasted_array = array1 + broadcast_value
20 print(broadcasted_array)
21
22 # 4. Sorting the array
23 sorted_array = np.sort(array1)
24 print(sorted_array)
```

Average time

0.031 s

31.00 ms

Maximum time

0.037 s

37.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 37 ms

Debug

Expected output

1 1 1 2 2 2

Actual output

1 1 1 2 2 2

Value to search: 1

Value to search: 1

Value to count: 2

Value to count: 2

Value to add: 2

Value to add: 2

[0 1 2]

[0 1 2]

3

3

Terminal

Test cases

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject:** Count the number of students who scored 90 or more marks in each subject.

3.2.7. Student Data Analysis and Operations

- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored >=90:** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Operation...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4 # 1
5 print("All student Details:\n", a)
6 # 2
7 print("Total Students:", a.shape[0])
8 # 3
9 print("All Student Roll Nos", a[:, 0])
10 # 4
11 print("Subject 1 Marks", a[:, 1])
12 # 5
13 print("Min marks in Subject 2", np.min(a[:, 2]))
14 # 6
15 print("Max marks in Subject 3", np.max(a[:, 3]))
16 # 7
17 print("All subject marks:", a[:, 1:])
18 # 8
19 total = np.sum(a[:, 1:], axis=1)
20 print("Total Marks", total)
21 # 9 (ROUND FIX)
22 avg_student = np.round(np.mean(a[:, 1:], axis=1), 1)
23 print(avg_student)
24 # 10
25 avg_subject = np.round(np.mean(a[:, 1:], axis=0), 1)
26 print("Average Marks of each subject", avg_subject)
27 # 11
28 print("Average Marks of S1 and S2", avg_subject[:2])
29 # 12 (ARRAY FIX)
30 print("Average Marks of S1 and S3", np.array([avg_subject[0],
```

Terminal Test cases

< Prev Reset Submit Next >

3.2.7. Student Data Analysis and Operations

Operation...

```
31 avg_subject[2]))
32 # 13
33 print("Roll no who got maximum marks in Subject 3", a[np.argmax(a[:, 3]), 0])
34 # 14
35 print("Roll no who got minimum marks in Subject 2", a[np.argmin(a[:, 2]), 0])
36 # 15 (SHAPE FIX)
37 print("Roll no who got 24 marks in Subject 2", a[a[:, 2] == 24][:, 0])
38 # 16
39 print("Count of students who got marks in Subject 1 < 40", np.sum(a[:, 1] < 40))
40 # 17
41 print("Count of students who got marks in Subject 2 > 90:", np.sum(a[:, 2] > 90))
42 # 18
43 print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=0))
44 # 19 (FIXED ORDER)
45 print("Roll no:", a[:, 0])
46 print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=1))
47 # 20
48 print(np.sort(a[:, 1]))
49 # 21
50 print(a[(a[:, 1] >= 50) & (a[:, 1] <= 90)])
51 print(a)
52 # 22
53 print(np.where(a[:, 1] == 79))
```

Terminal Test cases

< Prev Reset Submit Next >

Explorer

Average time
0.040 s
40.00 ms

Maximum time
0.040 s
40.00 ms

1 out of 1 shown test case(s) passed

Test case 1 40 ms

Debug

Expected output	Actual output
All Student Details:-	All Student Details:-
[[301,67,77,88]]	[[301,67,77,88]]
[[302,78,88,77]]	[[302,78,88,77]]
[[303,45,56,89]]	[[303,45,56,89]]
[[304,88,98,45]]	[[304,88,98,45]]
[[305,78,88,99]]	[[305,78,88,99]]
[[306,68,78,36]]	[[306,68,78,36]]
[[307,79,12,45]]	[[307,79,12,45]]
[[308,46,45,77]]	[[308,46,45,77]]
[[309,89,32,88]]	[[309,89,32,88]]
[[310,79,24,33]]	[[310,79,24,33]]
Total Students:-10	Total Students:-10
All Student Roll Nos [301,302,303,304,305,306,307,308,309,310]	All Student Roll Nos [301,302,303,304,305,306,307,308,309,310]
Subject 1 Marks [67,78,45,88,78,68,79,46,89,79]	Subject 1 Marks [67,78,45,88,78,68,79,46,89,79]
Min marks in Subject 2 12.0	Min marks in Subject 2 12.0
Max marks in Subject 3 99.0	Max marks in Subject 3 99.0
All subject marks:-[[67,77,88]]	All subject marks:-[[67,77,88]]
[[78,88,77]]	[[78,88,77]]
[[45,56,89]]	[[45,56,89]]
[[88,98,45]]	[[88,98,45]]
[[78,88,99]]	[[78,88,99]]
[[68,78,36]]	[[68,78,36]]
[[79,12,45]]	[[79,12,45]]
[[46,45,77]]	[[46,45,77]]

< Prev

Reset

Submit

Next >